

INSTITUTO TECNOLÓGICO DE JIQUILPAN

Ingeniería en Sistemas Computacionales



DOCUMENTO DE ARQUITECTURA

Agente Conversacional Mínimo — Sistema de Agentes Inteligentes

PROYECTO DE IA AGÉNTICA

Docente: Martínez González Leonardo

Entregable 1 — Fundamentos y funciones del Agente

Integrante	Rol
Omar Vega López	Equipo de Desarrollo
Angel de Jesús Canela Figueroa	Equipo de Desarrollo
Isamar Chávez Díaz	Equipo de Desarrollo
Katia Paola Victoria Escalera	Equipo de Desarrollo

1. Objetivo del Proyecto

Desarrollar, en equipo, un sistema de agentes inteligentes para dominio libre que integre orquestación de agentes (LangChain / LangGraph), backend de microservicios, persistencia mediante base de datos vectorial y despliegue en contenedores, gestionado con metodología Scrum en JIRA. Esta primera entrega establece las bases teóricas, el repositorio colaborativo y un agente conversacional mínimo funcional.

2. Información General

Proyecto	Agente Conversacional Mínimo
Materia	Proyecto de IA Agéntica
Repositorio	github.com/Omarvl-XCoder/Agente-Conversacional-Minimo
Gestión ágil	JIRA — Espacio "Agente Conversacional" (Scrum)
Entrega	1 de 4 — Fundamentos y funciones del Agente

3. Modelo del Agente Inteligente (PEAS)

Para diseñar el agente se aplicó el modelo PEAS (Percepción, Efectores, Ambiente, Sensores), estándar para caracterizar agentes racionales:

Percepción	Mensajes de texto enviados por el usuario a través de la interfaz web.
Efectores	Respuesta en lenguaje natural generada por el modelo de lenguaje y mostrada en el chat.
Ambiente	Conversacional, basado en texto; parcialmente observable (no conoce el estado real del usuario, solo lo que este escribe); dinámico en el contenido de la conversación; episódico por sesión, con memoria de corto plazo dentro de esa sesión.
Sensores	Petición HTTP (POST) que llega al endpoint /chat del microservicio FastAPI, con el mensaje del usuario en el cuerpo (body) de la solicitud.

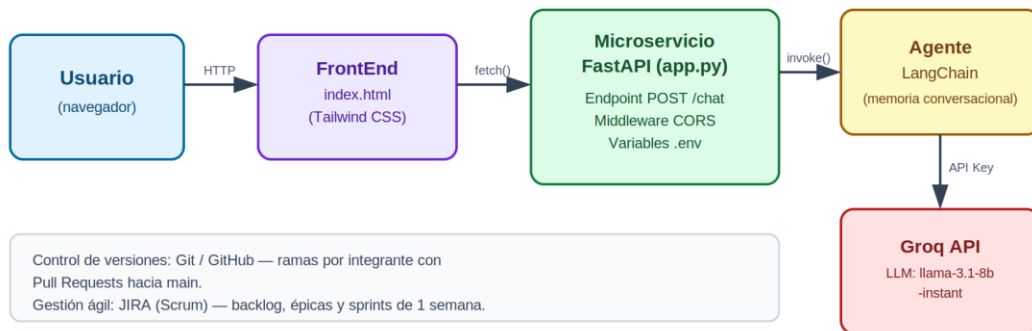
3.1 Tipo de agente

Se optó por un agente reactivo basado en modelos, con memoria de corto plazo. A diferencia de un agente puramente reactivo (que solo responde al estímulo actual), este agente utiliza el historial de la conversación (gestionado por LangChain) para mantener contexto entre mensajes dentro de una misma sesión. No es aún un agente deliberativo/planificador ni un Sistema Multiagente (SMA): no planifica secuencias de acciones ni consulta

una base de conocimiento externa. Estas capacidades se incorporarán en entregas posteriores (RAG en la Entrega 2, orquestación multiagente con LangGraph en la Entrega 4).

4. Arquitectura del Sistema

El sistema está compuesto por dos microservicios desacoplados (FrontEnd y Backend) que se comunican vía HTTP, y un servicio externo de inferencia (Groq API):



4.1 Componentes

- FrontEnd (index.html): interfaz de chat construida con HTML y Tailwind CSS. Envía el mensaje del usuario mediante `fetch()` al backend y muestra la respuesta.
- Microservicio Backend (FastAPI — `app.py`): expone el endpoint `POST /chat`, incluye middleware de CORS para permitir peticiones del navegador, y carga la API Key de Groq desde variables de entorno (`.env`).
- Agente (LangChain): recibe el mensaje desde FastAPI, mantiene la memoria conversacional de la sesión y arma la petición hacia el modelo de lenguaje.
- Groq API (LLM llama-3.1-8b-instant): servicio externo de inferencia que genera la respuesta en lenguaje natural.
- `cliente.py`: script auxiliar usado para probar el endpoint `/chat` desde consola, sin depender del FrontEnd.

5. Heurística y Búsqueda Aplicada

En esta entrega el agente responde de forma directa (una sola herramienta: el modelo de lenguaje), por lo que no requiere aún búsqueda heurística. Sin embargo, se identificó dónde será necesaria en próximas entregas: cuando el agente cuente con varias herramientas disponibles (retriever RAG, motor de reglas, técnica de IA elegida en la Entrega 3), tendrá que decidir cuál invocar según el tipo de consulta. Ahí se aplicará una función

heurística de enrutamiento (costo/beneficio estimado de cada herramienta) y, en la Entrega 4, se formalizará como búsqueda en espacio de estados (DFS/BFS) sobre el flujo de decisiones del agente.

6. Flujo de Trabajo con Git

Para evitar romper la rama principal, el equipo sigue este flujo:

- `git pull origin main` — traer los cambios más recientes antes de empezar.
- `git checkout -b nombre-de-la-tarea` — crear una rama propia por integrante/tarea.
- `git add . / git commit -m "..."` — guardar avances locales.
- `git push -u origin nombre-de-la-tarea` — subir la rama a GitHub.
- Pull Request hacia main, revisado por el integrante que administra el repositorio antes de hacer merge.

Ejemplo real: el Pull Request #1 corrigió el nombre del modelo en Backend/app.py, sustituyendo un identificador no disponible en Groq (meta-llama/llama-prompt-guard-2-86m) por llama-3.1-8b-instant.

7. Gestión Ágil — Evidencia en JIRA

El proyecto se gestiona en JIRA bajo metodología Scrum, en el espacio "Agente Conversacional", con sprints de una semana.

7.1 Sprint 1 (21–26 de septiembre) — Cerrado

Clave	Historia / Tarea	Estado
SCRUM-5	Configuración de infraestructura y repositorio colaborativo Git	Finalizado
SCRUM-1	Investigación teórica de IA y teorías de la inteligencia	Finalizado
SCRUM-10	Agente conversacional mínimo funcional	Finalizado

Objetivo del sprint: establecer las bases teóricas de IA, formalizar el modelo PEAS y la arquitectura inicial del agente, y configurar el repositorio colaborativo en Git con exclusión de credenciales.

7.2 Sprint 2 (26 de septiembre – 2 de octubre) — En curso

Clave	Historia / Tarea	Estado
SCRUM-7	Integración de LangChain con ChatGroq y memoria conversacional	Finalizado
SCRUM-8	Construcción del endpoint POST /chat y pruebas con cliente en consola	Finalizado

SCRUM-9	Documento de arquitectura + backlog en JIRA con 2 sprints cerrados	Finalizado
SCRUM-11	FrontEnd para el Agente	Finalizado

7.3 Capturas de pantalla — Tablero JIRA

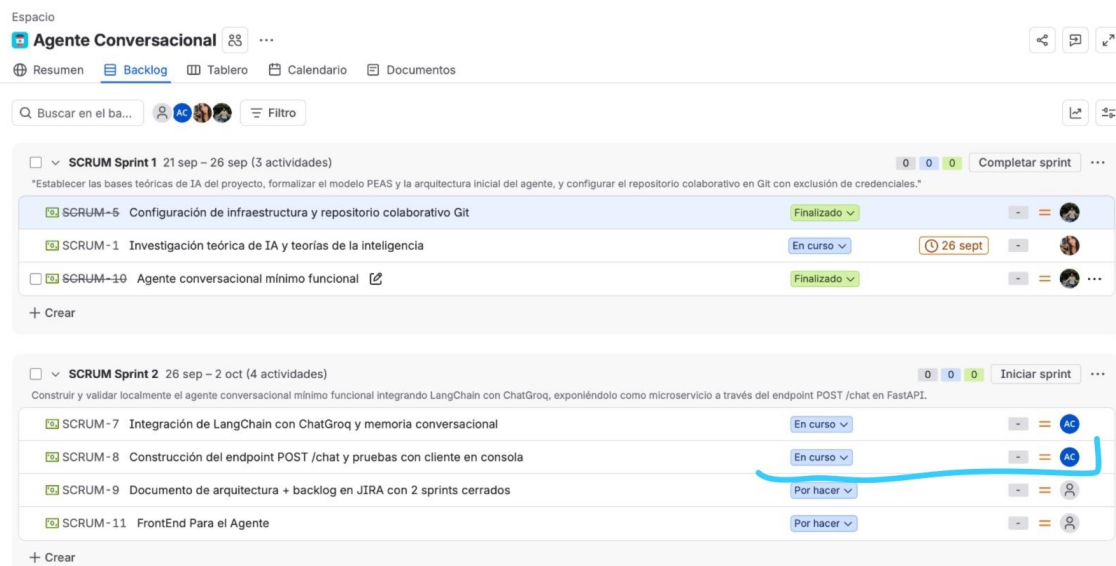


Figura 1. Backlog de JIRA — Sprint 1 finalizado y Sprint 2 planificado.

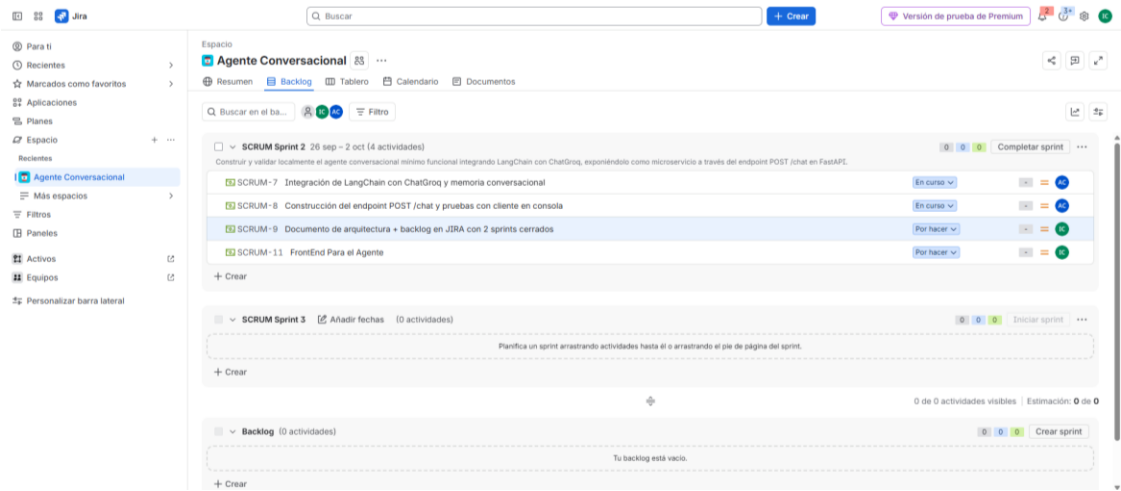


Figura 2. Sprint 2 activo, con SCRUM-9 y SCRUM-11 asignados y en curso.

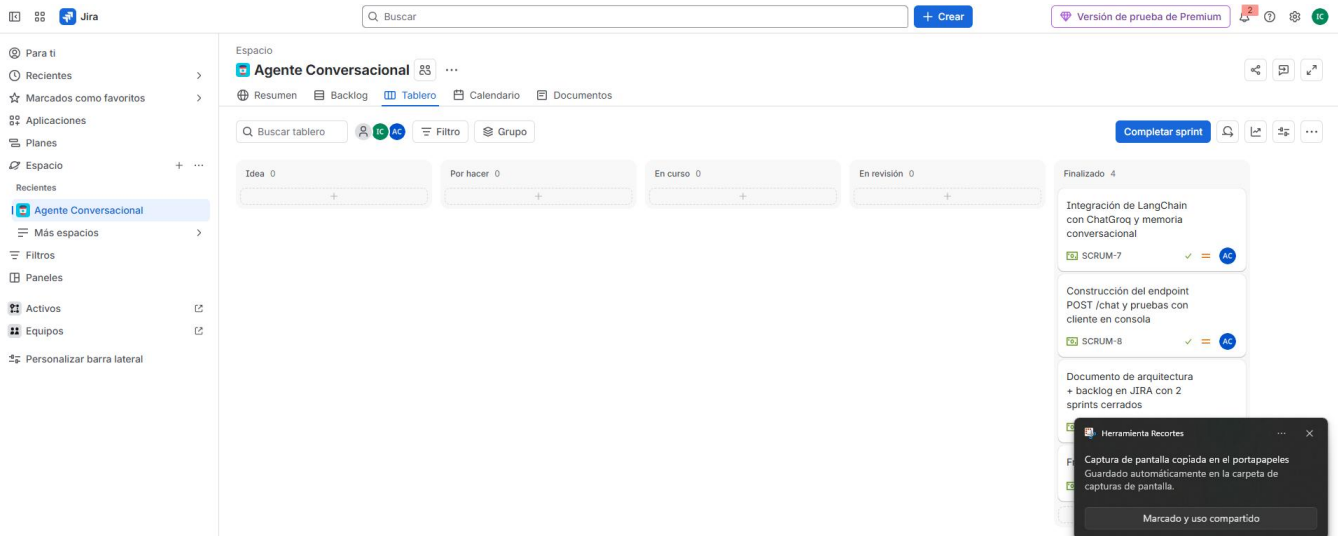


Figura 3. Sprints finalizados

8. Evidencia de Funcionamiento Local

El agente fue ejecutado localmente (backend con Uvicorn en `http://127.0.0.1:8000` y FrontEnd abierto en el navegador), respondiendo correctamente a los mensajes del usuario:

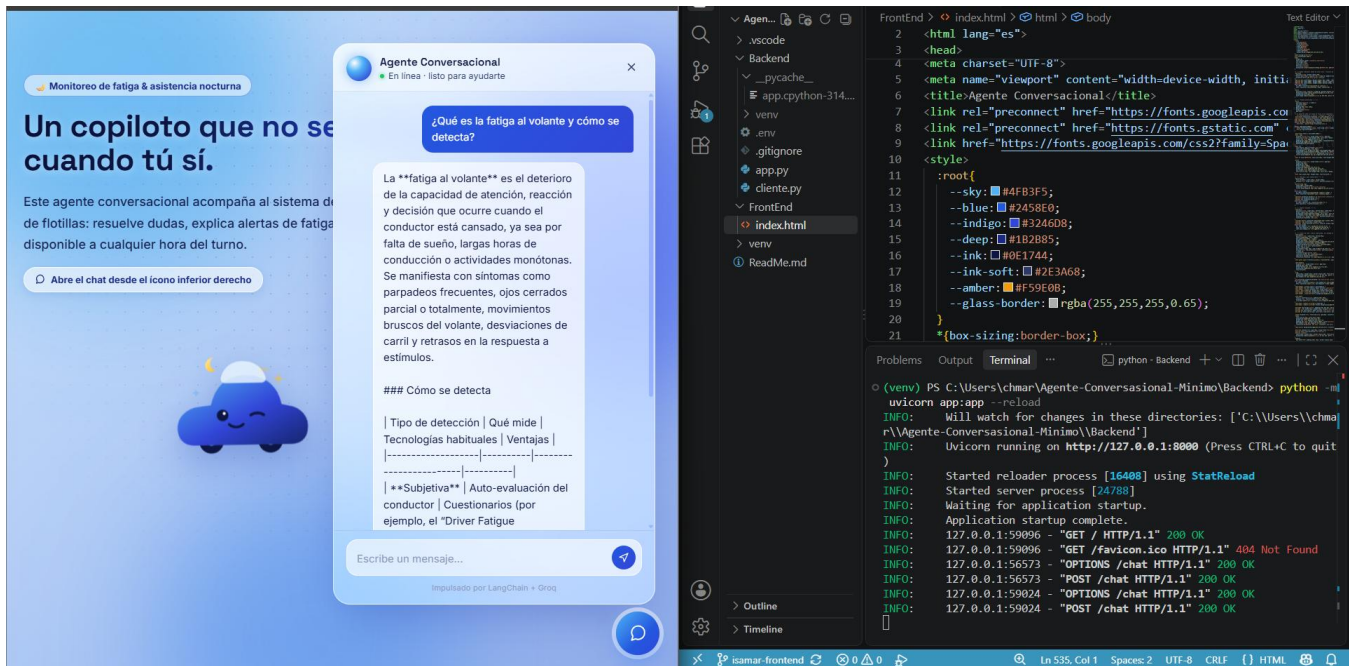


Figura 4. Agente conversacional respondiendo en local, backend levantado con Uvicorn.

9. Conclusión

Con esta primera entrega quedan establecidas las bases del proyecto: un repositorio Git colaborativo con flujo de ramas y Pull Requests, un agente conversacional mínimo funcional basado en LangChain y Groq, expuesto como microservicio FastAPI a través del endpoint /chat, y un backlog gestionado en JIRA con 2 sprints cerrados/en curso. Estas bases permitirán incorporar en las siguientes entregas representación del conocimiento (RAG), razonamiento probabilístico, una técnica adicional de IA y, finalmente, orquestación multiagente con despliegue en producción.